

A Probabilistic Choreography Language for PRISM

Marco Carbone¹[0000–0001–9479–2632] and Adele Veschetti²[0000–0002–0403–1889]

¹ IT University of Copenhagen
maca@itu.dk

² Technische Universität Darmstadt
adele.veschetti@tu-darmstadt.de

Abstract. We present a choreographic framework for modelling and analysing concurrent probabilistic systems based on the PRISM model-checker. This is achieved through the development of a choreography language, which is a specification language that allows to describe the desired interactions within a concurrent system from a global viewpoint. Employing choreographies provides a clear and comprehensive view of system interactions, enabling the discernment of process flow and detection of potential errors, thus ensuring accurate execution and enhancing system reliability. We equip our language with a probabilistic semantics and then define a formal encoding into the PRISM language and discuss its correctness. Properties of programs written in our choreographic language can be model-checked by the PRISM model-checker via their translation into the PRISM language. Finally, we implement a compiler for our language and demonstrate its practical applicability via examples drawn from the use cases featured in the PRISM website.

This work is currently under submission for publication.

1 Introduction

Understanding and programming distributed systems pose difficult challenges due to their inherent complexity and the potential for elusive edge cases within their intricate interactions. Unlike monolithic systems, distributed programs involve multiple nodes operating concurrently and communicating over networks, introducing a multitude of potential failure scenarios and nondeterministic behaviours. One of the primary challenges in understanding distributed systems lies in the fact that the interactions between multiple components can diverge from the sum of their individual behaviours. This emergent behaviour often results from subtle interactions between nodes, making it difficult to predict and reason about the system’s overall behaviour.

PRISM [1] is a probabilistic model checker that offers a specialised language for the specification and verification of probabilistic concurrent systems. PRISM has been used in various fields, including multimedia protocols [7], randomised distributed algorithms [4, 6], security protocols [9, 5], and biological systems [2, 3]. At its core, PRISM provides a declarative language with a set of constructs

for describing probabilistic behaviours and properties within a system. Given a distributed system, we can use PRISM to model the behaviour of each of its nodes, and then verify desired properties for the entire system. However, this approach can become difficult to manage as the number of nodes increases.

Choreographic programming [8] is an emerging programming paradigm in which programs, referred to as choreographies, serve as specifications providing a global perspective on the communication patterns inherent in a distributed system. In particular, instead of relying on a central orchestrator or controller to dictate the behavior of individual components, choreographic languages focus on defining communication patterns and protocols that govern the interactions between entities. In essence, choreographies abstract away the internal details of individual components and emphasise the global behaviour as a composition of decentralised interactions. This approach facilitates the automatic generation of decentralised implementations that are inherently correct-by-construction.

This work presents a choreographic language designed for modelling concurrent probabilistic systems. Additionally, we introduce a compiler capable of translating protocols described in this language into PRISM code. This choreographic approach not only simplifies the modelling process but also ensures integration with PRISM's powerful analysis capabilities.

As an example, consider the following simplified version of a system presented in the PRISM documentation³, where a user moves between different states (0, 1, or 2) based on certain events (α, β, γ) with corresponding rates (λ, μ, θ), and where a checkout process transitions between two states (0 or 1).

```

1  ctmc
2  module User
3      User_STATE : [0..2] init 0;
4
5      [alpha_1] (User_STATE=0) → lambda : (User_STATE'=1);
6      [alpha_2] (User_STATE=0) → lambda : (User_STATE'=2);
7      [beta] (User_STATE=1) → mu : (User_STATE'=0);
8      [gamma_1] (User_STATE=2) → theta : (User_STATE'=1);
9      [gamma_2] (User_STATE=2) → theta : (User_STATE'=2);
10 endmodule
11
12 module CheckOut
13     CheckOut_STATE : [0..1] init 0;
14
15     [alpha_1,alpha_2] (CheckOut_STATE=0) → 1 : (CheckOut_STATE'=1);
16     [beta] (CheckOut_STATE=1) → 1 : (CheckOut_STATE'=0);
17     [gamma_1,gamma_2] (CheckOut_STATE=1) → 1 : (CheckOut_STATE'=1);
18 endmodule

```

In PRISM, modules are individual processes whose behaviour is specified by a collection of commands, in a declarative fashion. Processes have a local state, can interact with other modules and query each other's state. Above, the modules User (lines 2-10) and CheckOut (lines 12-18) synchronise on labels `alpha_1`, `alpha_2`, `beta`, `gamma_1` and `gamma_2`. On line 5, `(User_STATE=0)` is a condition indicating that this transition is enabled when `User_STATE` has value 0. The variable `lambda` represents a rate, since the program models a Continuous

³ <https://www.prismmodelchecker.org/casestudies/thinkteam.php>

Time Markov Chain (CTMC). The command (`User_STATE'=1`) is an update, indicating that `User_STATE` changes to 1 when this transition fires.

Understanding the interactions between processes in this example might indeed be challenging, especially without additional context or explanation. Alternatively, when formalised using our choreographic language, the same model becomes significantly clearer.

```
C0 := User → Check : (+["lambda*1"] ; C1 +["lambda*1"] ; C2)
C1 := User → Check : (+["beta*1"] ; C0)
C2 := User → Check : (+["mu*1"] ; C1 +["mu*1"] ; C2)
```

In this model, we define three distinct choreographies, namely `C0`, `C1`, and `C2`. These choreographies describe the interaction patterns between the modules `User` and `Check`. The state updates resulting from these interactions are not explicitly depicted as they are not relevant for this particular protocol, but necessary in the PRISM implementation. As evident from this example, the choreographic language facilitates a straightforward understanding of the interactions between processes, minimizing the likelihood of errors.

Through our contributions, we aim to provide a smooth workflow for modeling, analyzing, and verifying concurrent probabilistic systems, ultimately increasing their usability in various application domains. In particular, by employing choreographies, we gain a clear and comprehensive view of the interactions occurring within the system, allowing us to discern the flow of processes and detect any potential sources of error in the modelling phase.

Contributions. Our contributions can be categorised as:

- We propose a choreographic language equipped with well-defined syntax and semantics, tailored specifically for describing concurrent systems with probabilistic behaviours. To the best of our knowledge, this is the first probabilistic choreography language that is not a type abstraction.
- Then, we introduce a semantics for PRISM, based on its original semantics [1].
- Furthermore, we establish a rigorous definition for a translation function from choreographies to PRISM, and address its correctness. This translation (projection) serves as a crucial intermediary step in transforming (compiling) models described in our choreographic language into PRISM-compatible representations.
- Lastly, we implement a compiler that translates choreographies into PRISM, allowing users to use PRISM’s robust analysis features while benefiting from the expressiveness of our choreographic language.

Acknowledgments. The work is partially funded by H2020-MSCA-RISE Project 778233 (BEHAPI), by the ATHENE project "Model-centric Deductive Verification of Smart Contracts", and by the Carlsberg Foundation.

References

1. Prism documentation. <https://www.prismmodelchecker.org/>, accessed: 2023-09-05
2. Dannenberg, F., Kwiatkowska, M., Thachuk, C., Turberfield, A.: DNA walker circuits: computational potential, design, and verification. In: Soloveichik, D., Yurke, B. (eds.) Proc. 19th International Conference on DNA Computing and Molecular Programming (DNA 19). LNCS, vol. 8141, pp. 31–45. Springer (2013)
3. Heath, J., Kwiatkowska, M., Norman, G., Parker, D., Tymchyshyn, O.: Probabilistic model checking of complex biological pathways. In: Priami, C. (ed.) Proc. Computational Methods in Systems Biology (CMSB’06). Lecture Notes in Bioinformatics, vol. 4210, pp. 32–47. Springer Verlag (2006)
4. Kwiatkowska, M., Norman, G., Parker, D.: Probabilistic verification of herman’s self-stabilisation algorithm. *Formal Aspects of Computing* **24**(4), 661–670 (2012)
5. Kwiatkowska, M., Norman, G., Parker, D., Vigliotti, M.: Probabilistic mobile ambients. *Theoretical Computer Science* **410**(12–13), 1272–1303 (2009)
6. Kwiatkowska, M., Norman, G., Segala, R.: Automated verification of a randomized distributed consensus protocol using Cadence SMV and PRISM. In: Berry, G., Comon, H., Finkel, A. (eds.) Proc. 13th International Conference on Computer Aided Verification (CAV’01). LNCS, vol. 2102, pp. 194–206. Springer (2001)
7. Kwiatkowska, M., Norman, G., Sproston, J.: Probabilistic model checking of deadline properties in the IEEE 1394 FireWire root contention protocol. *Formal Aspects of Computing* **14**(3), 295–318 (2003)
8. Montesi, F.: *Introduction to Choreographies*. Cambridge University Press (2023)
9. Norman, G., Shmatikov, V.: Analysis of probabilistic contract signing. *Journal of Computer Security* **14**(6), 561–589 (2006)